1.5 Rekursive Algorithmen

Ruft sich eine Methode selbst mit anderem Parameter auf, ist sie **rekursiv**. *Website: Station 1.5, Labor und Projekt*

Aufgabe 1 *Hin und zurück*

```
void countdown(int n) {
   if (n == 0) {
      println("Start!");
   } else {
      println("hin " + n);
      countdown(n - 1);
      println("zurück " + n);
   }
}
```

a) Sage vorher, was `countdown(3)` ausgibt:

_____

_____

b) Wenn „Start!“ erscheint, warten _____ Aufrufe. Als Erstes gibt _____ „zurück“ aus.

Rekursive Algorithmen

Drei Fragen: ① **Abbruchbedingung**: Wann steht die Antwort sofort fest? ② **Verkleinern**: Mit welchem kleineren Problem ruft sich die Methode auf? ③ **Zusammensetzen**: Was macht sie mit der Antwort?

Wartende Aufrufe liegen auf dem _____, der zuletzt aufgerufene wird als _____ fertig.

Linear: höchstens _____ rekursiver Aufruf je Durchlauf, Darstellung: _____. **Verzweigt:** _____ rekursive Aufrufe je Durchlauf, Darstellung: _____.

Aufgabe 2 *Wenn die Rekursion nicht endet (Projekt, Klasse Experiment)*

a) Starte jede Fassung mit `n = 3` und notiere die ersten fünf Werte von `n`. **b)** Welche der drei Fragen ①–③ ist verletzt? Begründe kurz.

	Kern des Codes	a) erste Werte von `n`	b) verletzt, weil …
A	`(kein if) fassungA(n - 1)`		
B	`if (n == 0) … fassungB(n - 2)`		
C	`if (n == 0) … fassungC(n)`		

c) TODO 2: Fassung B endet für jede Startzahl mit `if (` _____ `)`. Jeder Aufruf muss der Abbruchbedingung _____, sonst bricht Java mit einem _____ ab.

Aufgabe 3 *Aufruftabelle zum ggT*

```
int ggT(int a, int b) {
   if (b == 0) {
      return a;
   }
   return ggT(b, a % b);
}
```

a) Fülle die Tabelle für `ggT(84, 36)` aus. Rückgaben von unten nach oben!

Aufruf	a	b	a % b	Abbruch?	Rückgabe
`ggT(84, 36)`	84	36			

b) `ggT(36, 84)` ruft als Erstes _____ auf: Die Parameter werden _____. Insgesamt sind es _____ Aufrufe.

Aufgabe 4 *Lineare Rekursion planen und programmieren (TODO 4a–c)*

a) Plane jede Methode mit den drei Fragen, bevor du sie im Projekt programmierst. Schreibe Java-Ausdrücke.

Methode	① Abbruch: Bedingung → Antwort	② rekursiver Aufruf	③ zusammensetzen
`quersumme(n)`			
`umrechnen(n, basis)`			
`umdrehen(s)`			

b) Trage die Werte von `n` in allen Aufrufen und das Ergebnis ein (Labor oder Projekt):

`umrechnen(22, 2)`: n = _____ Ergebnis _____

`umrechnen(22, 3)`: n = _____ Ergebnis _____

Je größer `basis`, desto _____

Aufgabe 5 *Fibonacci: verzweigte Rekursion (TODO 5a, 5b)*

Definition: $fib(0) = 0,\ fib(1) = 1,\ fib(n) = fib(n-1) + fib(n-2)$ für $n \geq 2$. Folge: 0, 1, 1, 2, 3, 5, 8, …

a) Ergänze den Code.

```
int fib(int n) {
   aufrufe++;                    // TODO 5b
   if (_____________ ) { return 0; }
   if (n == 1) { return _______ ; }
   return ____________________________ ;
}
```

b) Zeichne den Aufrufbaum von `fib(4)` und schreibe jede Rückgabe an ihren Aufruf.

Die beiden `if` sind die

_____.

`fib` ist verzweigt rekursiv, weil

_____

c) Zähle mit dem Projekt (Abschnitt Aufgabe 5):

n	5	10	15	20	25
fib(n)					
Aufrufe					

Wird `n` um 5 größer, steigt die Zahl der Aufrufe etwa auf das _____-Fache.

d) In `fib(5)` wird `fib(2)` _____-mal berechnet.

e) Was passiert bei `fib(-1)`? Begründe.

_____

_____

_____

Aufgabe 6 *Der Tresor: vier Algorithmen in Reihe (TODO 6)*

Notiere die Zwischenwerte. `fib(17)` = _____ → Quersumme _____ → im Zweiersystem _____ → rückwärts gelesen (Code) _____

Die Nachricht im Tresor lautet: _____

_____